

Bootstrap 5

Rev 2.5 del 05/01/2026

Introduzione a Bootstrap	2
Terminali, Containers e Grid System	3
Dimensioni del testo	6
Box Dimensions	6
Allineamenti	7
Margin and Padding	8
Colors	9
Buttons	10
Forms e controlli.....	11
Icons (bootstrap 4)	12
Icons (bootstrap 5)	13
Immagini.....	14
Tables	14
Gli attributi data-bs	15
Alert	16
Toast	17
Modals	18
Navbar	19
Card	20
Collapse	20
Jumbotron	21
Applicazione d un tema dark / light	21

Bootstrap 5

Bootstrap è un framework `css` e `Javascript` per la creazione di pagine web di tipo **responsive** (adattate) cioè in grado di adattarsi alle dimensioni dello schermo sul quale verranno visualizzate. Utilizza un approccio di tipo **mobile-first**, cioè orientato alla programmazione mobile. Sviluppato per creare **Twitter** e poi rilasciato come progetto open source.

Versioni

Dal sito ufficiale getbootstrap.com / **download** si può scaricare l'ultima versione.

La versione **4.0** è stata rilasciata il 10 agosto 2017. Tra le altre cose, è stata eliminata la cartella **fonts** contenente le glyphicons (che erano distribuite come file di caratteri nei formati ttf, eot, svg e woff). Per l'utilizzo delle glyphicons si rimanda ora al sito **awesome**, aggregato al progetto bootstrap

La versione **5.0** è stata rilasciata il 5 maggio 2021. E' stata completamente rimossa la dipendenza da jQuery, sostituito con JavaScript puro. Le icone sono di nuovo parte integrante di bootstrap ma inserite in un file separato : **bootstrap-icons.min.css**

Link delle librerie (bootstrap5)

Si può utilizzare direttamente il **CDN** server di bootstrap (bootstrapcdn.com) o altri.

Occorre linkare:

```
css/bootstrap.min.css
js/bootstrap.bundle.min.js
font/bootstrap-icons.min.css
```

La versione **bootstrap.bundle.min.js** congloba al suo interno i riferimenti a Popper.js e Owl Carousel (necessario soltanto se si usano componenti di tipo Carousel per scorrere un set di immagini tramite le classiche frecce laterali).

Se si utilizza in locale, per l'utilizzo delle icone, occorre scaricare anche **bootstrap-icons.min.css** ed i due files

```
bootstrap-icons.woff
bootstrap-icons.woff2
```

da inserire in una apposita sottocartella **fonts**

Il metatag viewport

Per poter utilizzare al meglio bootstrap ogni pagina html deve includere, prima di ogni altra cosa, i seguenti metatag:

```
<meta name="viewport"
      content="width=device-width, initial-scale=1">
```

Il meta-name **viewport** definisce le dimensioni della finestra in fase di apertura e il valore di zoom. I valori impostati fanno in modo che il browser adatti automaticamente la dimensione della pagina alle dimensioni dello schermo. `initial-scale=1` imposta il valore di zoom iniziale (100%)

`user-scalable=no` impedisce all'utente di effettuare zoom sulla pagina

I Terminali

Bootstrap suddivide i terminali in 5 categorie (6 in bootstrap v5) identificate tramite appositi **breakpoint** che indicano le dimensioni dello schermo:

- | | |
|---|--|
| • xs (extra-small for smaller phones) | terminali con larghezza < 576 px |
| • sm (small for phones) | terminali con larghezza < 768 px |
| • md (medium for tablets) | terminali con larghezza < 992 px |
| • lg (large for desktops) | terminali con larghezza < 1200 px |
| • xl (extra-large for larger desktops) | terminali con larghezza < 1400 px |
| • xxl (extra extra-large) | terminali con larghezza > 1400 px |

i Container

I **containers** sono delle classi che costituiscono la base del **Grid System** e che **consentono al loro interno di creare righe e colonne**. Ogni riga si estende per tutta la larghezza del container

A seconda del terminale su cui la pagina viene visualizzata, la larghezza del **container** assume valori diversi in modo da lasciare un po' di spazio a sinistra e a destra centrando il container rispetto al genitore. La classe **.container** (che è la più utilizzata) assegna al contenitore una larghezza fissa così definita:

- | | |
|------------------------|---------|
| • terminale xs | 100% |
| • terminale sm | 540 px |
| • terminale md | 720 px |
| • terminale lg | 960 px |
| • terminale xl | 1140 px |
| • terminale xxl | 1320 px |

Ogni riga viene poi automaticamente suddivisa in **12 celle** adattative tutte con la stessa larghezza ottenuta come **larghezzaContainer / 12**

Per cui quando si allargano le dimensioni dello schermo, **ogni volta che scatta un breakpoint si allargano anche le dimensioni delle singole celle**. Nel caso dalla classe **.container**

- In caso di terminale **extra-small** ogni cella occupa l'intera larghezza del terminale. Il contenuto della cella che eccede le dimensioni della stessa viene nascosto
- In caso di terminale **sm** ogni cella assume una larghezza di **540/12 = 45** px.
- In caso di terminale **md** ogni cella assume una larghezza di **720/12 = 60** px
- In caso di terminale **lg** ogni cella assume una larghezza di **960/12 = 80** px
- In caso di terminale **xl** ogni cella assume una larghezza di **1140/12 = 95** px.
- In caso di terminale **xxl** ogni cella assume una larghezza di **1320/12 = 110** px

Se si desidera che, in un certo contesto, il container NON si allarghi oltre una certa soglia, il modo più semplice e veloce è quello di utilizzare la seguente regola CSS:

```
.details {  
  max-width: 720px;           /* dimensione schermo md */  
  margin: 10px auto;  
}
```

Il contenitore **.details**, anche su schermi molto grandi, non si allargherà mai oltre le dimensioni del contenitore md (720px)

Bootstrap

In realtà esistono diversi altri tipi di contenitori, indicati nella seguente tabella (dal sito ufficiale) :

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
<code>.container</code>	100%	540px	720px	960px	1140px
<code>.container-sm</code>	100%	540px	720px	960px	1140px
<code>.container-md</code>	100%	100%	720px	960px	1140px
<code>.container-lg</code>	100%	100%	100%	960px	1140px
<code>.container-xl</code>	100%	100%	100%	100%	1140px
<code>.container-fluid</code>	100%	100%	100%	100%	100%

La prima colonna riporta i **vari tipi di contenitori** disponibili in bootstrap, che sono essenzialmente 3:

container-fluid

Nel container-fluid non ci sono breakpoint. Occupa **sempre** l'intero spazio a sua disposizione (**100% del genitore**), indipendentemente dal tipo di terminale. La larghezza delle colonne si adatta in modo **fluid** rispetto alle dimensioni del contenitore. Stringendo il contenitore si stringono anche le celle. Negli schermi grandi occupa però l'intera pagina con un effetto scadente. Può andare bene per eventuali contenitori interni, ma **MAI** per il contenitore principale !

container-{breakpoint}

Via di mezzo tra `.container` e `.container-fluid`. Ogni colonna occupa il 100% della larghezza a sua disposizione fino a quando non si raggiunge il breakpoint indicato. Ad esempio nel caso del **container-lg** (che è il preferibile)

- la riga occuperà il 100% della larghezza a sua disposizione nel caso di terminali xs sm md (cioè come il `container-fluid`)
- la riga occuperà invece la larghezza prefissata nel caso di terminali lg e xl (cioè come il `container` normale)

Note

- E' possibile utilizzare in sezioni diverse della stessa pagina contenitori diversi, ognuno con il proprio grid system. Ognuno può avere un suo tipo (fluid o non fluid).
- All'interno di un container è possibile annidare altri container

Modificatori di cella

La classe `.row` aggiunge all'elemento una **width:100%** rispetto al contenitore.

Una row può contenere al suo interno come figli diretti soltanto celle.

I cosiddetti **modificatori di cella** (es `col-md-3`, `col-xl-6` etc) consentono di modificare la larghezza di una cella accorpono più celle in orizzontale (tipo `rowspan` dell'html) in modo da creare celle più larghe. La somma deve comunque sempre fare 12. *In assenza di modificatori le celle vengono comunque visualizzate e ad ogni cella verrà assegnato il valore **auto** che indica il numero minimo di colonne necessarie alla visualizzazione del contenuto presente nella cella.*

```
<div class="container">
  <h2>container</h2>
  <div class='row'>
    <div class='col-md-3'> </div>
    <div class='col-md-6'> </div>
    <div class='col-md-3'> </div>
  </div>
</div>
```

Nell'esempio, sui terminali **medium**

- la prima cella occupa 3 colonne
- la seconda 6 colonne
- la terza 3 colonne.

Nel momento in cui si imposta un valore su una singola classe :

- **il valore impostato verrà applicato su tutti i terminali di dimensione maggiore o uguale laddove non sia stata indicata una diversa classe di livello superiore.**
- **Nei terminali di dimensioni inferiore, in assenza di una apposita impostazione, le celle collassano, cioè vengono automaticamente visualizzate una sotto l'altra a piena larghezza**

Nell'esempio precedente le tre celle assumono la dimensione indicata nel caso di terminali **md**, **lg** e **xl**.

Nel caso invece di terminali inferiori (**xs** e **sm**) le celle vengono visualizzate una sotto l'altra.

Utilizzo parallelo di più modificatori

Le varie classi possono essere utilizzate **parallelamente** per ottimizzare il layout rispetto ai vari dispositivi, nel senso che ogni singolo oggetto potrà occupare un numero diverso di celle a seconda del contesto. Es:

```
<div class="row">
  <div class="col-sm-9 col-md-7" > </div>
  <div class="col-sm-3 col-md-5" > </div> </div>
```

Su una finestra di tipo **sm** il primo DIV occupa 9 colonne, il secondo 3 colonne.

Su una finestra di tipo **md** (o superiori) il primo DIV occupa 7 colonne, il secondo 5 colonne.

Siccome le classi relative a schermi maggiori 'coprono' le impostazioni delle classi relative a schermi minori, occorre scrivere le classi in ordine di terminale crescente.

Note sulle dimensioni delle celle

- Se si impostano ad esempio le classi **sm** ed **lg**, i terminali **sm** e **md** utilizzeranno la suddivisione "sm", mentre i terminali **lg** e **xl** utilizzeranno la suddivisione "lg".
Nei terminali **xs** le celle invece collassano e vengono visualizzate una sotto l'altra.
- Se si imposta la classe **md-12** l'oggetto sarà visualizzato a 12 colonne.
- Le classi **col-sm-0 col-md-0 col-lg-0 col-xl-0** non esistono.
- Le classi **col-xs-n** non esistono ed hanno come nome semplicemente **col-n**.
Se si impostano, le celle verranno comunque adattate tramite il solito schema a 12 colonne.
I valori assegnati tramite questi modificatori valgono ovviamente anche per tutti i terminali superiori (a meno che non venga ridefinito un altro apposito modificatore). Nei terminali **xs** vengono rimosse tutte la spaziature fra una cella e l'altra in modo da ottimizzare l'occupazione dello spazio
I modificatori **xs** sono comunque abbastanza poco utilizzati perché sui terminali più piccoli di solito è bene avere le celle una sotto l'altra a piena larghezza.
- L'ultima cella, se vuota, può facoltativamente essere omessa. Bootstrap la aggiungerà in automatico estendendola per un numero di colonne tale per cui la somma complessiva arrivi a 12.

Utilizzo dei modificatori generici

Sono disponibili anche dei modificatori generici, cioè privi dell'indicazione del numero di colonne.

col-xs col-sm col-md col-lg col-xl

Se si applicano alle celle questi modificatori, lo spazio verrà diviso equamente fra le colonne impostate.

Ad esempio se si hanno 3 celle e si mette **lg-7** sulla prima cella e poi **lg lg** sulle altre due:

- la prima cella occuperà 7 colonne,
- le rimanenti 5 colonne verranno equamente suddivise fra le altre due celle. Cioè ognuna delle due celle occuperà 2.5 colonne, risultato NON ottenibile con i qualificatori tradizionali.

gutter

Il “**gutter**” indica lo **spazio tra le colonne** all'interno di una **griglia** e serve a separare visivamente i contenuti in colonne senza dover usare padding o margini manuali.

Può essere applicato solo insieme alla classe `.row`. Ogni `.row` ha un gutter orizzontale (spazio tra le colonne) e un gutter verticale (spazio tra le righe). Di default, in Bootstrap 5 il gutter orizzontale è 1.5rem (**g-3** vale a dire 24px), ma può essere modificato con le seguenti classi:

g-0	Nessun gutter (sia orizzontale che verticale)
g-1 ... g-5	Gutter crescente (sia orizzontale che verticale)
gx-0 ... gx-5	Gutter orizzontale specifico
gy-0 ... gy-5	Gutter verticale specifico

Offset fra le colonne

Per ogni colonna è possibile specificare un offset, inteso come numero di colonne vuote da aggiungere davanti alla colonna corrente che, di conseguenza, verrà spostata verso destra.

```
<div class="col-sm-6 col-sm-offset-3"> </div>
```

Questa classe è molto comoda nel caso in cui si voglia definire ad esempio una unica cella centrata di larghezza ad esempio 6. L'utilizzo dell'attributo `col-sm-offset-3` consente di evitare di scrivere la prima cella. Omettendo anche l'ultima cella la scrittura diventa estremamente compatta.

Annidamento delle colonne

E' possibile suddividere una colonna in sottocolonne annidate sempre con somma 12.

Le sottocolonne devono a loro volta essere sempre inserite all'interno di una row

```
<div class="row">
  <div class="col-sm-8">
    <div class="row">
      <div class="col-sm-6"> 4 colonne complessive (8/2) </div>
      <div class="col-sm-6"> 4 colonne complessive (8/2) </div>
    </div>
  </div>
</div>
<div class="col-sm-4"> </div>
```

Dimensioni del testo

fs-1	2.5rem	~40px	
fs-2	2rem	~32px	
fs-3	1.75rem	~28px	
fs-4	1.5rem	~24px	
fs-5	1.25rem	~20px	
fs-6	1rem	~16px	(default)

Box Dimensions

w-25	
w-50	Larghezza in % rispetto al genitore
w-75	
w-100	
h-25	
h-50	Altezza in % rispetto al genitore
h-75	
h-100	

Allineamenti

Allineamento Orizzontale delle singole celle

Le classi

text-start
text-center
text-end

Consentono di allineare orizzontalmente il testo interno ad una cella. Vanno scritti sul tag della cella

E' anche possibile agire sul contenuto testuale di una cella :

text-lowercase
text-uppercase
font-weight-bold

Allineamento Verticale delle singole celle

Le classi

align-self-start
align-self-center
align-self-end

Consentono di allineare verticalmente il testo interno ad una cella. Vanno scritti sul tag della cella

```
<div class="row">
  <div class="col align-self-start">
    One of three columns
  </div>
  <div class="col align-self-center">
    One of three columns
  </div>
  <div class="col align-self-end">
    One of three columns
  </div>
</div>
```

Allineamento Verticale dell'intera riga

Le classi

align-items-start
align-items-center
align-items-end

applicabili soltanto ai tag `<div class="row">` consentono di allineare verticalmente il contenuto dell'intera row in alto, in basso oppure al centro

```
<div class="row align-items-start">
  <div class="col">
    One of two columns
  </div>
  <div class="col">
    One of two columns
  </div>
</div>
```

Riposizionamento orizzontale delle celle

Quando non si specificano tutte le celle di una riga (cioè quando la somma non fa 12) è possibile definire la posizione delle celle inesistenti utilizzando, sulla riga, le seguenti classi:

```
<div class="row justify-content-start">
  <div class="col-4">
    One of two columns
  </div>
  <div class="col-4">
    One of two columns
  </div>
</div>
```

justify-content-start	lascia 4 colonne vuote a destra (default)
justify-content-end	lascia 4 colonne vuote a sinistra
justify-content-center	lascia 2 colonne vuote a sinistra e due a destra
justify-content-between	lascia 4 colonne vuote al centro
justify-content-around	lascia 1 colonna vuota a sx, 1 a dx, 2 in centro

Margin and Padding

Per margini e padding si utilizzano apposite classi scritte nel seguente formato:

{property}{sides}-{breakpoint}-{size}

dove :

property può assumere i valori

m = margin
p = padding

sides può assumere i valori

t - for classes that set margin-top or padding-top
b - for classes that set margin-bottom or padding-bottom
s - (start) for classes that set margin-left or padding-left in LTR, margin-right or padding-right in RTL
e - (end) for classes that set margin-right or padding-right in LTR, margin-left or padding-left in RTL
x - for classes that set both *-left and *-right
y - for classes that set both *-top and *-bottom
blank - for classes that set a margin or padding on all 4 sides of the element

breakpoint può assumere i soliti valori sm md lg xl (e maggiori)

Se non è indicato significa xs

size può assumere i valori

0 - for classes that eliminate the margin or padding by setting it to 0
1 - for classes that set the margin or padding to \$spacer * .25
2 - for classes that set the margin or padding to \$spacer * .5
3 - for classes that set the margin or padding to \$spacer
4 - for classes that set the margin or padding to \$spacer * 1.5
5 - for classes that set the margin or padding to \$spacer * 3
auto - for classes that set the margin to auto

dove \$spacer rappresenta il valore di default di margin / padding utilizzato dal browser in uso, cioè in pratica equivale al **rem**. Mezzo spacer equivale **0.5 rem**

Bootstrap

Esempi:

```

class="mb-5"           margin-bottom = 5 spacer
class="py-sm-1.5"      Applica un padding y di 1.5 per schermi >= sm
class="ms-md-3"        Applica un margine sinistro (start) per schermi >= md

```

mx-auto esegue la centratura orizzontale (come margin:auto)

Colors

Colori relativi al testo

Sono disponibili le seguenti classi per definire il colore di un testo:

<code>.text-muted</code>	// grigio chiaro	This text is muted.
<code>.text-primary</code>	// blu	This text is important.
<code>.text-success</code>	// verde	This text indicates success.
<code>.text-info</code>	// celeste	This text represents some information.
<code>.text-warning</code>	// giallo	This text represents a warning.
<code>.text-danger</code>	// rosso	This text represents danger.
<code>.text-secondary</code>	// grigio chiaro	Secondary text.
<code>.text-dark</code>	// grigio scuro	Dark grey text.
<code>.text-light</code>	// grigio chiarissimo	Body text.
<code>.text-white</code>	// bianco	
<code>.text-body</code>	// nero	

Colori relativi allo sfondo

Sono disponibili le seguenti classi per definire il colore di uno sfondo:

<code>.bg-primary</code>	// blu (con testo bianco)
<code>.bg-success</code>	// verde
<code>.bg-info</code>	// celeste
<code>.bg-warning</code>	// giallo
<code>.bg-danger</code>	// rosso
<code>.bg-secondary</code>	// grigio chiaro
<code>.bg-dark</code>	// grigio scuro
<code>.bg-light</code>	// quasi bianco

This text is important.

This text indicates success.

This text represents some information.

This text represents a warning.

This text represents danger.

Secondary background color.

Dark grey background color.

Light grey background color.

Buttons

Oltre alla classe base **btn** che aggiunge semplicemente un padding sui 4 lati, in bootstrap 4.x.x sono disponibili le seguenti classi :

Basic Primary Secondary Success Info Warning Danger Dark Light Link

```
<button type="button" class="btn">Basic</button>
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
<button type="button" class="btn btn-info">Info</button>
<button type="button" class="btn btn-warning">Warning</button>
<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-dark">Dark</button>
<button type="button" class="btn btn-light">Light</button>
<button type="button" class="btn btn-link">Link</button>
```

Queste classi possono essere applicate ai tag `<a>` `<input type=button>` `<button>`

In parallelo può essere applicata una delle seguenti classi che assegna una specifica dimensione al pulsante (la differenza comunque è di pochi px)

- **.btn-xs**
- **.btn-sm**
- **.btn-md** (default)
- **.btn-lg**
- **.btn-xl**

che consentono di impostare in modo più fine larghezza e altezza del pulsante (sempre che il pulsante non implementi le classi `form-control` o `btn-block` nel qual caso la larghezza coprirà tutta l'area disponibile)

La classe **disabled** visualizza il pulsante come disabilitato

La classe **active** visualizza il pulsante come premuto (con sfondo un po' più scuro).

La classe btn-block

Il pulsante per default utilizza una larghezza pari alla larghezza del suo contenuto

Se il pulsante è inserito all'interno di un grid system, **btn-block** fa sì che il pulsante si estenda per l'intera lunghezza delle celle all'interno delle quali è inserito. Eliminato in bootstrap 5. Sostituito con:

```
<button type="button" class="btn btn-primary d-grid gap-2"> Invia </button>
```

Attenzione però che se sul pulsante si applica la classe **input-group**, il pulsante, pur in presenza di **btn-block**, utilizzerà soltanto lo spazio necessario al proprio contenuto.

Button Outline

Bootstrap 4.x.x fornisce i seguenti buttons outline, dotati di un semplice bordino colorato :

Primary Secondary Success Info Warning Danger Dark Light

```
<button type="button" class="btn btn-outline-primary">Primary</button>
<button type="button" class="btn btn-outline-secondary">Secondary</button>
<button type="button" class="btn btn-outline-success">Success</button>
<button type="button" class="btn btn-outline-info">Info</button>
```

Forms e Controlli

Bootstrap fornisce tre possibili layout per le form:

- **vertical-form** (default)
- **inline-form** (class="form-inline") // eliminato in bootstrap 5
- **horizontal-form** (class="form-horizontal")

Nel caso di **vertical form** tutti i controlli interni alla form vengono visualizzati uno sotto l'altro, indipendentemente da form-group (ogni controllo occupa l'intera riga)

Nel caso di **inline form** tutti i controlli interni alla form vengono visualizzati su una stessa riga, indipendentemente da form-group

Nel caso di **horizontal-form** tutti i controlli interni alla classe **form-group** vengono visualizzati sulla stessa riga, mentre form-group diversi vengono visualizzati su righe successive.

form-group

Tutti i gruppi di controlli interni ad una form dovrebbero implementare una classe **form-group** che funge da "contenitore" per i controlli. In realtà si limita ad aggiungere un margine inferiore di 15px in modo da distanziare leggermente i vari gruppi.

La classe **form-group** diventa significativa se abbinata alla classe **horizontal-form**.

*Questa classe non può essere applicata ad un tag che implementa la classe **row** la quale **deve** essere applicata ad un elemento dedicato.*

form-control

La classe **form-control** trasforma un controllo in un controllo di form stilizzato secondo bootstrap. Inoltre fa sì che il controllo si estenda per l'intera larghezza a sua disposizione. Se non si desidera che il controllo si allarghi al 100% è possibile applicare in parallelo una classe **w-** che sovrascrive la classe form-control. Ad esempio w-50. Oppure impostare una larghezza fissa da CSS

input-group

La classe **input-group** consente di estendere un Text Box aggiungendo una icona o un testo o un pulsante prima o dopo del TextBox medesimo.

L'elemento da aggiungere davanti / dietro implementa solitamente la classe **input-group-text**

```
<form>
  <div class="col-sm-4">
    <div class="form-group input-group">
      <label for="email" class="input-group-text">Username:</label>
      <input type="email" class="form-control" id="email">
    </div>
  </div>
</form>
```

is-invalid

Su tutti i controlli che implementano una classe base form-control, form-select, ecc. è disponibile una classe **is-invalid** che consente di ricolorare di rosso il bordo del controllo in cui sia stato inserito un valore non valido.

La classe badge

La classe badge viene utilizzata per creare etichette o indicatori di piccole dimensioni che di solito servono a mostrare numeri, stati o contatori vicino a testi o elementi della UI.

```
<span class="badge bg-success"> Si </span>  
<span class="badge bg-danger"> No </span>
```

Crea due etichette (che sembrano dei pulsanti non cliccabili) una verde ed una rossa con all'interno il testo indicato

Le icone in bootstrap 4

w3school / icons. Da bootstrap 4 le icone non fanno più parte del pacchetto base ma devono essere scaricate ed installate separatamente. In alternativa è possibile usare le icone di fontawesome.com/icons

A tal fine occorre scaricare la libreria awesome oppure impostare un link ad un CDN.

All'inizio del sito c'è un comodissimo campo di search che consente di ricercare le icone desiderate

Le icone users sono contenute nel gruppo Users e People

Le icone password sono contenute nel gruppo Security

Per inserire una icona si utilizza il tag **<i>** :

```
<i class="fas fa-user"></i>
```

La prima classe indica il tipo di icona:

fas = icona solida

far = icona regolare stilizzata

fal = icona light con i bordi più sottili

fad = icona duotone in due colori

Per centrare e spaziare adeguatamente l'icona occorre "avvolgerla in un tag **** con associata la classe **input-group-text** che ha sostituito la classe **input-group-addon** di bootstrap 3.x.x. A differenza di **input-group-addon** questa nuova classe non consente però di ricolorare l'icona in caso di errore.

```
<div class="form-group input-group">  
  <span class="input-group-text">  
    <i class="fas fa-user"> </i>  
  </span>  
  <input type="text" class="form-control" id="usr" placeholder="Username">  
</div>
```

	Email
	Password

Le icone in bootstrap 5

`<i class="bi bi-list fs-2"> </i>`

bi-emoji-frown-fill	faccina triste
bi-list	burger button
bi-search	lente
bi-trash	cestino
bi-star	stellina
bi-star-fill	stellina piena (nera)
bi-person	user stilizzato
bi-person-circle	
bi-person-fill	più evidente e moderno
bi-people-fill	
bi-lock	password
bi-lock-fill	più evidente e moderno
bi-shield-lock	per comunicare protezione
bi-envelope →	busta per mail
bi-envelope-fill	busta per mail fill
bi-envelope-at →	busta con @
bi-envelope-at-fill	busta con @ fill
bi-calendar →	calendario generico
bi-calendar-date →	calendario con giorno
bi-calendar-event →	calendario evento
bi-cake →	torta (spesso usata per compleanno)
bi-exclamation-triangle	
bi-book →	singolo libro aperto
bi-collection-play	raccolta di Film
bi-journals →	raccolta di Libri
bi-boombox-fill	raccolta di audiocassette
bi-pc-display	pc + display

La classe fs-n che controlla le dimensioni del testo può essere applicata anche alle icone

L'icona assume il colore del genitore in cui si trova. Per cambiare il colore:

```
icon.classList.add("bi", "bi-star", "text-danger") // oppure
icon.style.color="red"
```

Immagini

Le immagini non hanno una classe predefinita necessaria per poter applicare le classi aggiuntive, che possono invece essere applicate direttamente al tag IMG.

Le classi aggiuntive applicabili al tag IMG sono le seguenti :

.img-fluid	visualizza l'immagine in tutta la sua grandezza, indipendentemente da width, adattandosi però automaticamente alle dimensioni dello schermo o alle dimensioni del genitore. L'immagine si riduce automaticamente con la riduzione del genitore.
.rounded	definisce spigoli arrotondati.
.rounded-circle	rende l'immagine ovale. Le dimensioni rimangono quelle indicate da width
.img-thumbnail	aggiunge un bordo swl colore indicato (ad esempio border-primary) con all'interno un padding bianco. border-5 consente di definire lo spessore del bordo Inoltre arrotonda un pochino gli spigoli .
.float-end	rende l'immagine fluttuante a destra
.float-start	rende l'immagine fluttuante a sinistra
.float-md-end	rende l'immagine fluttuante solo su schermi >= md

Nota: Per poter inserire le immagini nel contesto a righe e colonne, in bootstrap4 occorre inserire l'immagine all'interno di un tag DIV in quanto non era consentito applicare al tag IMG le classi relative alle colonne (ad esempio la classe `class="col-sm-6"`). In bootstrap5 questo è invece possibile.

```

```

Tables

La classe **.table** applicata ad un tag html **table** applica uno stile "base" della tabella in cui le righe hanno sfondo bianco e bordo orizzontale di separazione. Le colonne non hanno bordo.

In assenza della classe **.table** le classi interne non funzionano correttamente.

Se si utilizza la classe **.table**, occorre utilizzare anche i tag **THEAD** e **TBODY**, altrimenti le classi bootstrap non funzionano correttamente.

Insieme a **.table**, sul tag **<table>** può essere applicata una delle seguenti classi:

.table-striped	le righe vengono visualizzate con sfondo alternato, una a sfondo bianco ed una a sfondo grigio chiaro.
.table-hover	viene aggiunto un effetto di roll over sulle righe. Al passaggio del mouse su una riga, lo sfondo assumerà una tonalità leggermente più scura Può essere usata in abbinamento alla precedente.
.table-dark	applica un colore di sfondo dark. Può essere utilizzata in abbinamento a table-striped per ottenere uno sfondo alternato di grigio e dark.
.table-bordered	viene aggiunto un bordo anche alle colonne che normalmente NON hanno bordo
.table-borderless	Vengono eliminati tutti i bordi anche sulle righe
.table-sm	rende la tabella più compatta dimezzando tutti i padding
.table-responsive	se la larghezza della tabella eccede il container applica una barra di scorrimento

Colori di sfondo

E' possibile assegnare un colore specifico di sfondo all'intera tabella o ad ogni singola cella.
I colori utilizzabili sono i soliti colori relativi agli sfondi.

```
<table class="table table-success table-striped table-hover ">
  <thead>
    <tr class="table-primary">
      <th>Firstname</th>
      <th>Lastname</th>
      <th>Email</th>
    </tr>
  </thead>
  <tbody>
    <tr class="table-secondary">
```

Le classi **thead-color** consentono di applicare un colore di sfondo all'intera thead

```
<thead class="thead-dark">
```

.table-responsive

Da applicare su un tag **DIV esterno** alla tabella, rende la tabella **responsive**, nel senso che compare automaticamente una scroll bar quando lo schermo assume una larghezza inferiore a 992px

```
<div class="table-responsive">
  <table class="table">
    ...
  </table>
</div>
```

Gli attributi data-bs-*

Gli attributi data-bs-* sono attributi HTML personalizzati usati da Bootstrap 5 che dicono a Bootstrap come deve comportarsi un componente senza scrivere manualmente codice javascript. Sono letti dal codice JavaScript di Bootstrap.

Cosa significa data-bs-* ?

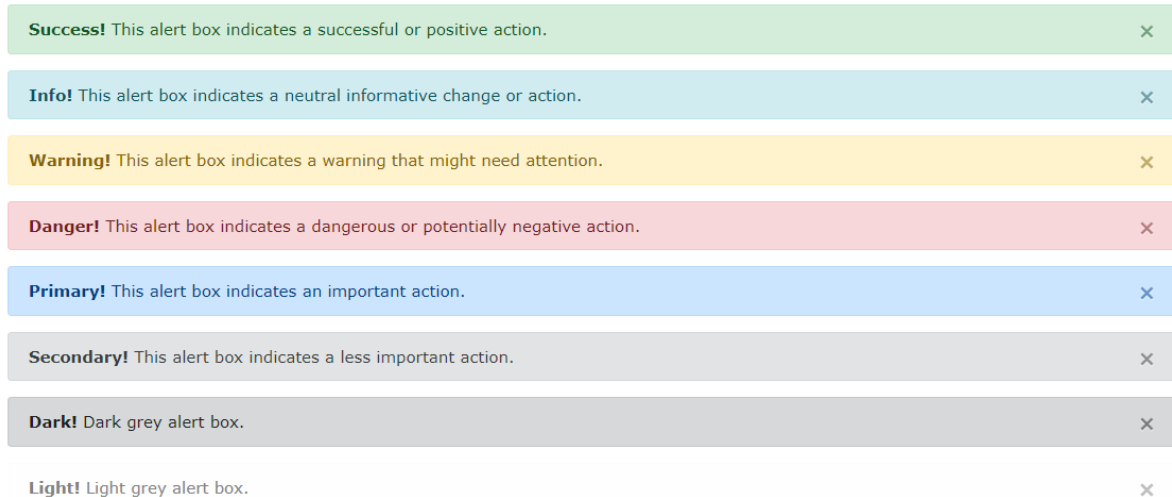
data-* → standard HTML per attributi personalizzati
bs → abbreviazione di Bootstrap

Esempio:

```
<button
  data-bs-toggle="modal"
  data-bs-target="#myModal">
  Apri modal
</button>
```

alert

Una alert è semplicemente un DIV interno ad un qualsiasi contenitore che di solito è inizialmente nascosto e viene visualizzato per segnalare un messaggio



```
<div class="alert alert-danger d-none alert-dismissible my-2 text-center">
  <i class="bi bi-exclamation-triangle"></i>
  Login fallito. Riprova più tardi.
  <button class="btn-close" data-bs-dismiss="alert"></button>
</div>
```

d-none = display:none; Il tag è inizialmente nascosto

alert-dismissible = Predisporre l'alert a ospitare un pulsante X di chiusura posizionandolo correttamente sulla destra in maniera inline

data-bs-dismiss="alert" dice a Bootstrap che questo bottone ha la funzione di **dismiss**, cioè nascondere / rimuovere l'elemento con classe .alert più vicino. In realtà questo attributo non solo nasconde la alert, ma la rimuove completamente dal DOM, per cui se la alert deve poi essere riaperta **questo attributo NON DEVE essere utilizzato** ed occorre gestire la chiusura da javascript.

fade show = Per avere un effetto di transizione all'avvio, l'alert non deve avere d-none. Deve avere invece la classe **fade** (nascosto tramite opacità) e poi, tramite JavaScript, si aggiunge la classe **show** per farlo apparire sfumando

role="alert" riguarda le specifiche **ARIA** (Accessible Rich Internet Applications) e serve per migliorare l'accessibilità per utenti ipovedenti.

Per visualizzare/nascondere la alert da javascript :

```
const alert = document.getElementById("idAlert");
alert.classList.remove("d-none"); // visualizza
alert.classList.add("d-none");    // nasconde
```

Per rimuovere definitivamente una alert dal DOM tramite javascript (in alternativa alla classe **data-bs-dismiss**) occorre prima creare una istanza la quale rende disponibile il metodo .close()

```
const alert = document.getElementById("idAlert");
const w = new bootstrap.Alert(alert);
w.close();
```

Toast

E' simile alla alert ma scompare automaticamente dopo un certo tempo. Esempio :

```
function showToast(message) {
  const toast = document.createElement('div');
  toast.className = `toast align-items-center text-white bg-primary border-0`;
  toast.setAttribute('role', 'alert');

  // Toast Body
  const flex = document.createElement('div');
  flex.className = 'd-flex';
  toast.appendChild(flex);

  const body = document.createElement('div');
  body.className = 'toast-body';
  body.textContent = message;
  flex.appendChild(body);

  // Pulsante Opzionale, per eventuale chiusura anticipata
  const button = document.createElement('button');
  button.type = 'button';
  // la classe me-2 prevale a destra rispetto a m-auto perché all'interno
  // di Bootstrap è scritta dopo
  button.className = 'btn-close btn-close-white me-2 m-auto';
  button.setAttribute('data-bs-dismiss', 'toast');
  flex.appendChild(button);

  // Append to existing container
  document.getElementById('toastContainer').appendChild(toast);

  // instance
  const _toast = new bootstrap.Toast(toast, { delay: 3000 });
  _toast.show();
}
```

Se non si specifica il delay, viene applicato un default di 5 secondi

Modals

Una finestra modale è una finestra che viene aperta ‘sopra’ rispetto alla pagina corrente.
Può essere bloccante o non bloccante

```
<div class="modal fade" id="myModal" tabindex="-1" >
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Titolo</h5>
        <button class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <p>Testo della modale</p>
      </div>
      <div class="modal-footer">
        <button class="btn btn-danger"
          data-bs-dismiss="modal"> Chiudi </button>
        <button class="btn btn-success"> Salva </button>
      </div>
    </div>
  </div>
</div>
```

fade = applica un effetto fade in apertura e chiusura della finestra

data-bs-dismiss="modal" , a differenza della alert, questo attributo questa volta esegue semplicemente una hide(), per cui la modale può poi essere riaperta senza problemi.

Un eventuale pulsante **salva** va gestito da javascript

Visualizzazione di una modale

// Invece di leggere il puntatore alla modale, occorre istanziarla (una sola volta all'avvio)

```
const modal = new bootstrap.Modal("#myModal")
```

// Poi si settano i campi e si visualizza la modale

```
const span = document.getElementById("span1")
```

```
span.textContent = "my text"
```

```
modal.show();
```

// In corrispondenza del salva si chiude a modale

```
modal.hide() // chiude senza deallocare. Equivale a data-bs-dismiss
```

Visualizzazione ‘bloccante’ e/o a centro schermo:

```
<div class="modal fade" id="" data-bs-backdrop="static">
  <div class="modal-dialog modal-dialog-centered">
```

static la rende “bloccante”

centered la visualizza a centro schermo

navbar

```
<nav class="navbar bg-primary navbar-dark navbar-expand-md px-2 fixed-top">
  <div class="container-fluid">
    <!-- icon and title-->
    <a class="navbar-brand" href="#">
      <i class="bi bi-collection-play me-2"></i>
      Film Library
    </a>
    <!-- Burger Button che compare quando si restringe la pagina -->
    <button class="navbar-toggler" data-bs-toggle="collapse"
      data-bs-target="#navbarContent">
      <span class="navbar-toggler-icon"></span>
    </button>
    <!-- buttons -->
    <div class="collapse navbar-collapse" id="navbarContent">
      <ul class="navbar-nav me-auto mb-2 mb-lg-0">
        <li class="nav-item">
          <a class="nav-link active" href="#">Home</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="#">Details</a>
        </li>
      </ul>
      <!-- 2° contenitore automaticamente allineato a destra -->
      <div class="d-flex">
        <!-- icona amici -->
        <i class="bi bi-people-fill me-2" id="i-friends"></i>
        <!-- icona search -->
        <i class="bi bi-search" id="i-search"></i>
      </div>
    </div>
  </div>
</nav>
```

navbar-dark significa che la navbar ha uno sfondo scuro per cui i testi avranno colore chiaro
navbar-expand-md rende la navbar **collassabile**. Viene cioè visualizzata per intero su schermi $\geq$ md e collassata su schermi più piccoli.

fixed-top la rende fissa in alto. Attenzione che in questo il body della pagina partirà a **0px dall'alto**, andando a finire sotto la navbar. In questi casi la documentazione suggerisce la seguente impostazione:

```
body { padding-top: 56px; }
```

<button class="navbar-toggler"> è il pulsante tipo burger button che verrà visualizzato in corrispondenza del collapse. In realtà la classe **navbar-toggler** serve soltanto a definire la grafica del pulsante che verrebbe in ogni caso visualizzato.

data-bs-toggle="collapse" rappresenta l'azione da eseguire in corrispondenza del click sul burger button, cioè quella di aprire il menù indicato nel successivo target

**** consente di aggiungere l'icona del burger button all'interno del button. Senza questo span, a menù collassato, si avrebbe un button vuoto.

Volendo si può sostituire lo **** con una icona personalizzata:

```
<i class="bi bi-list fs-1"> </i>
```

dropdown item

```
<li class="nav-item dropdown">
  <a class="nav-link dropdown-toggle" href="#" data-bs-toggle="dropdown">
    Categorie
  </a>
  <ul class="dropdown-menu">
    <li><a class="dropdown-item active" href="#">All</a></li>
    <li><a class="dropdown-item" href="#">Pop</a></li>
    <li><a class="dropdown-item" href="#">Rap</a></li>
    <li><a class="dropdown-item" href="#">Rock</a></li>
  </ul>
</li>
```

Card

E' semplicemente un box bordato con un certo padding. Se non si specificano altre classi viene implementato il "basic card" a sfondo bianco. Altrimenti si può aggiungere una delle solite classi per la definizione dello sfondo, ad esempio **bg-primary**

```
<div class="card">
  
  <div class="card-body">
    <h5 class="card-title"> Titolo </h5>
    <p class="card-text"> Contenuto della card </p>
    <a class="btn btn-secondary"> Compra </a>
  </div>
</div>
```

Insieme a `class="card">` si possono aggiungere le seguenti classi:

"shadow-lg"	Applica una ombra di tipo lg (cioè molto grande) intorno alla card
"border-0"	Toglie il bordo
"rounded-3"	spigolo arrotondato di 3px

Collapse

Assegnando questa classe ad un qualunque oggetto della pagina, questo diventa 'collapsible' cioè **inizialmente nascosto** e visualizzabile mediante un click su un pulsante o altro elemento della pagina.

Elemento nascosto:

```
<div class="collapse" id="myID">
```

Elemento 'controllante':

```
<div data-toggle="collapse"
      data-target="#idElementoDaControllare"> </div>
```

Il primo attributo indica che l'elemento corrente può controllare un elemento 'collapsible' della pagina. Il secondo attributo indica l'elemento da controllare. Nella stessa pagina possono esistere più elementi 'collapsible' controllati ognuno mediante un apposito elemento controllante.

La classe collapse non può essere applicata ad un elemento che implementa la classe **form-group** ma deve essere applicata ad un apposito elemento interno.

Jumbotron

Utilizzato per creare titoli con ampio risalto, sfondo grigio, spigoli arrotondati e font-size aumentato in modo da richiamare maggiormente l'attenzione del visitatore.

```
<div class="jumbotron">
  <h1>Bootstrap Tutorial</h1>
  <p>Bootstrap is the most popular framework for developing
      responsive, mobile-first projects on the web.</p>
</div>
```

Eliminato in bootstrap5 dove può essere sostituito con

class="p-4 mb-4 bg-light rounded-3"

Applicazione di un tema dark-light

```
$('html').attr('data-bs-theme', theme);
```

Applicando all'attributo **data-bs-theme** il valore del tema scelto, bootstrap 5 automaticamente applica il tema all'intera pagina. la variabile **theme** può assumere i seguenti valori:

- light
- dark
- auto // legge il tema di sistema

Il css di bootstrap sarà fatto in modo simile a questo (semplificato al massimo) :

```
[data-bs-theme="light"] {
  --bs-body-bg: #002b36;
  --bs-body-color: #839496;
}
```